



Open Distributed Systems — Bachelor's Thesis Proposal

A Use-Case-Driven Comparison of Layer-2 Scaling Approaches for IoT Micropayments and On-Chain Gaming

Ivan Ilin

Supervisor: Uwe A. Kuehn
Examiner: Prof. Dr. Manfred Hauswirth

June 4, 2026

Abstract

Public Ethereum sustains only around 15–30 transactions per second, which is insufficient for two emerging application classes: high-frequency IoT micropayments and real-time on-chain token gaming. Three Layer-2 paradigms — state channels, optimistic rollups, and ZK rollups — promise to close the throughput gap, but they sit in distinct research sub-communities and have not been benchmarked against each other under a shared workload. This thesis compares one representative of each family — Perun, Arbitrum Nitro, and zkSync Era — across the two named use cases on three metrics: latency to finality, deposit and on-chain transaction cost, and sustained throughput. Each protocol's lifecycle is decomposed into Open, Update, and Close sub-processes; all three protocols run against a single shared local Anvil L1 to eliminate public-testnet variance, and a single Python workload generator drives the protocol-under-test across a frequency sweep of {1 ms, 10 ms, 100 ms, 1 s}. The expected contribution is a comparison table of measured behaviours across the three protocols, the two use cases, and the three metrics, together with use-case-specific recommendations.

Contents

1 Motivation **3**

2 Research Question **4**

3 Related Work **4**

4 Methodology **5**
4.1 Sub-process decomposition 5
4.2 Use cases and workloads 6
4.3 Metrics 7
4.4 Hardware and setup 7

5 Deliverable and Timeline **8**

1 Motivation

Public blockchains such as Ethereum currently process around 15–30 transactions per second, which is insufficient for applications that demand sustained high-frequency payments or real-time interactivity [1]. Two such application classes motivate this thesis: high-frequency IoT micropayments and real-time on-chain token gaming. Layer-2 (L2) protocols address this throughput gap by moving most computation off-chain while still relying on the underlying L1 for security. Three architectural families dominate the current L2 landscape: state channels (e.g., Perun [2]), optimistic rollups (e.g., Arbitrum), and ZK rollups (e.g., zkSync Era). They make different trade-offs along the dimensions of latency, on-chain footprint, deposit requirements, liveness, and developer ergonomics, and the appropriate choice for a given application is not self-evident.

As the existing literature shows, most work covers either rollups or state channels in depth, but rarely both together in a single paper. Saif et al. (2024), a recent rollup-side survey, explicitly de-scope state channels and concentrate on rollup-based architectures only. Negka and Spathoulas (2021), a state-channel-side survey, conversely acknowledge that optimistic and zero-knowledge rollups have emerged as competitive alternatives, but discuss this only generically and without reference to any specific workload. To the best of our knowledge, no work compares the two families head-to-head against concrete applications.

Rebello et al. (2024) is a more recent and broader survey that does cover both paradigms in one paper, although the bulk of it focuses on payment channel networks (PCN) and rollups are addressed only in a single dedicated section near the end. Despite this imbalance, the paper closes with the hypothesis that payment channel networks are best suited to swift financial transactions and rollups to general-purpose applications. This hypothesis is not accompanied by an empirical comparison in the paper itself. The aim of this thesis is to test that hypothesis by measuring how three concrete implementations behave under the two use cases named above.

Such a comparison is more feasible today than it was a few years ago. EIP-4844 (March 2024) made rollup data storage cheaper through blob-carrying transactions, and stable reference implementations of all three families became available between 2022 and 2024 (Go-Perun, Arbitrum Nitro, zkSync Era). To the best of our knowledge, no published study has yet measured these three implementations against each other under specific application workloads.

Recent work from the TU Berlin Open Distributed Systems group by Kuehn demonstrates a Perun-based payment-service layer for an IoT data marketplace and shows that direct on-chain Ethereum cannot sustain transaction frequencies at 10 Hz, while a Perun off-chain channel sustains 100 Hz ($\approx$10 ms per micropayment) on resource-constrained hardware [3]. Kuehn's comparison is on-chain Ethereum vs. Perun only — no rollup paradigm is evaluated. This thesis extends that finding by asking whether the rollup paradigm (Arbitrum Nitro, zkSync Era) is competitive with Perun at the same frequencies and under the same use case.

State channels and rollups address the scaling problem through structurally different mechanisms, and this difference shapes the comparison that follows. A state channel takes the interaction of a fixed set of parties off-chain entirely: after an on-chain channel-opening transaction, the parties exchange signed state updates directly between themselves, and the L1 is involved again only on cooperative close or in case of a dispute. A rollup, by contrast, operates a shared L2 execution environment that any user can submit transactions to; the resulting L2 state is periodically committed back to L1 either with fraud-proof guarantees (optimistic rollups, e.g., Arbitrum) or with validity proofs (ZK rollups, e.g., zkSync). The two paradigms therefore differ both in trust model (bilateral peer agreement vs. a shared sequencer plus on-chain commitment) and in deployment footprint (one channel per interacting set vs. one shared rollup serving all users).

2 Research Question

> **For two use cases — high-frequency IoT micropayments and real-time on-chain token gaming — and the workloads derived from them, how do one state-channel implementation (Perun), one optimistic rollup (Arbitrum Nitro), and one ZK rollup (zkSync Era) compare on the metrics of (i) latency to finality, (ii) deposit and on-chain transaction cost, and (iii) sustained throughput? Under which conditions does each approach appear preferable?**

The question is intentionally narrow: three protocols, two use cases, three metrics — a finite comparison rather than an open-ended survey. The expected contribution is a comparison table of measured behaviours together with use-case-specific recommendations for which approach fits which application better.

3 Related Work

Anchor 1 — rollup side: Saif et al. (2024) [1]: a recent rollup-side survey in MDPI Future Internet. Its stated scope is restricted to rollup-based L2 solutions with respect to the data-availability problem: an opening section briefly sketches the broader L2 landscape (state channels, sidechains, plasma, rollups, validiums) in one comparative table, after which the paper narrows to rollups for the remaining $\sim$80 % of its content. It does not compare state channels and rollups along any application-specific dimension.

Anchor 2 — state-channel side: Negka and Spathoulas (2021) [4]: a state-channel-focused survey that decomposes state-channel lifecycles into Opening, Update, Dispute, and Closing sub-phases — a framing this thesis adopts and extends to rollups in Section 4.1. The same comparative analysis identifies blockchain-agnosticism among desirable state-channel features, which is relevant because Perun is blockchain-independent by design [2]. The paper does not engage with rollups beyond a single mention in its discussion section.

Hypothesis source — Rebello et al. (2024) [5]: a more recent IEEE COMST survey covering both paradigms in one paper, though weighted toward payment channel networks. Its closing hypothesis — referenced in Section 1 — that PCNs suit fast payments and rollups suit general-purpose applications is what this thesis tests empirically. Rebello et al. do not perform a workload-driven comparison.

Direct adjacent work — Kuehn (2025) [3]: a TU Berlin Open Distributed Systems group paper that proposes a Perun-based payment-service layer for an IoT data marketplace and measures it on a private Ethereum chain running on a Raspberry-Pi cluster. Kuehn shows that direct on-chain Ethereum cannot sustain 10 Hz micropayment rates, while a Perun off-chain channel sustains 100 Hz ($\approx$10 ms per payment) — and reports that an off-chain channel reduces network traffic in the underlying blockchain network. The comparison is on-chain Ethereum vs. Perun only — no rollup paradigm is evaluated. This thesis directly extends Kuehn's setting by adding Arbitrum Nitro and zkSync Era to the comparison at the same use case and frequency range.

Further adjacent works. Galal et al. (2019) [6] propose an efficient micropayment channel on Ethereum, with which Kuehn's extended Perun channel is comparable in intent. Gangwal et al. (2022) [7] is a broad L2 survey that organises the field into four protocol classes (channels, side/child chains, cross chains, and hybrid solutions) but does not benchmark them against application workloads. Sguanci et al. (2021) [8] is the closest prior framing with a use-case-driven angle (Lightning, Plasma, and Rollups on a supermarket-payment scenario); it predates EIP-4844 and uses Plasma rather than a modern state-channel framework. Avarikioti et al. (2025) [9] compare payment channels, rollups, and sidechains within a theoretical security framework, without application-level measurements.

4 Methodology

4.1 Sub-process decomposition

Because the three families differ in architecture, a single "complete process" comparison is not well defined. Following Negka and Spathoulas (2021), each protocol lifecycle is split into three sub-processes and the comparison is made at the sub-process level. Negka's separate Dispute phase is folded into Close here, since each protocol's dispute mechanism — Perun's adversarial-close path, Arbitrum's challenge window, zkSync's proof-verification delay — is invoked during close time rather than as a separate user-facing stage:

- **Open** / **Setup** — Perun: on-chain channel-funding transaction depositing tokens into the on-chain channel contract. Rollups: bridge-deposit transaction from L1 into the rollup.
- **Update** — Perun: off-chain signed state update between channel parties. Rollups: L2 transaction submitted to the sequencer.

- **Close** / **Settlement** — Perun: cooperative close (single on-chain tx) or adversarial close via the channel contract. Arbitrum: withdrawal back to L1, subject to the $\sim$7-day challenge window. zkSync: withdrawal back to L1, subject to the proof-verification delay.

4.2 Use cases and workloads

This thesis distinguishes between *use cases* and *workloads*. A **use case** names an application class: what the parties are doing and why. A **workload** specifies how that use case is exercised in measurement terms: transaction frequency, total duration and payload size per transaction. One use case can be exercised by several workloads.

Two use cases are studied:

- **UC1 — IoT micropayments.** A device pays a service for streamed sensor data or actuator commands. Small per-transaction value (sub-cent), with intermittent device connectivity.
- **UC2 — On-chain token gaming (two-player).** Two paired players exchange in-game ERC-20 token transfers as game actions resolve. Low per-transaction value, between the paired players. Extension to multi-player gaming with shared in-game economies is a natural follow-up and is left to future work.

For the measured comparison both use cases are run as 1-to-1 interactions. This is not a property of the use cases themselves — IoT micropayments in particular are not 1-to-1 by definition — but a constraint of the state-channel representative: `go-perun` channels are bilateral, so a fair like-for-like comparison against the rollups (which support 1-to-1 only as a special case of their native many-to-many model) is most cleanly made at the 1-to-1 level. Many-to-many topologies nonetheless remain relevant, and state channels can express them through virtual-channel and channel-network (routing) constructions [2]; this thesis will examine the many-to-many case at least theoretically, and practically if time allows.

UC3 (optional) — Truck platooning. Time permitting, a third use case is added: vehicle platooning, in which a convoy of trucks drives in close automated formation — the lead vehicle setting the speed and the followers maintaining short inter-vehicle gaps to reduce aerodynamic drag and increase road capacity. The vehicles continuously exchange coordination messages and settle accompanying micropayments, for instance followers compensating the lead truck for the drafting benefit, or vehicles paying to join or reorder within the platoon. Platooning is a strong stress test for the high-frequency end of the sweep, and can be framed either as a 1-to-1 extension of UC1 (each follower paying the lead) or, more naturally, as a many-to-many scenario across the convoy. It will be pursued only if the schedule allows.

For each use case, the workload is scripted as a (frequency, duration, payload-size) triple. This thesis sweeps the frequencies {1 ms, 10 ms, 100 ms, 1 s} per use case. The same frequency set is used for both use cases so that the protocol-vs-frequency behaviour can

be compared like-for-like across use cases. The assets exchanged are tokens (deployed via smart contracts), not native L1 coins.

4.3 Metrics

Each metric has a clear measurement procedure per protocol.

Metric 1 — Latency. Measured separately for each sub-process from Section 4.1.

- *Open*: seconds from broadcasting the opening transaction (channel-funding for Perun, bridge-deposit for the rollups) to its inclusion in an L1 block.
- *Update*: for Perun, the round-trip time of one signed off-chain message between two channel peers. For the two rollups, the time from submitting an L2 transaction to its confirmation by the sequencer (soft confirmation, not yet L1 finality).
- *Close*: for Perun, seconds from broadcasting the close transaction to L1 inclusion. For Arbitrum and zkSync, the additional time before funds become withdrawable on L1 ($\sim$7-day challenge window for Arbitrum, proof-verification delay for zkSync) is reported from protocol documentation rather than measured.

Metric 2 — Deposit and transaction cost. The *deposit* (rather than "capital lock-up") is the amount of value placed on-chain to open a channel or bridge into a rollup. The transaction cost of the deposit is the gas spent on the opening transaction itself. The primary figure reported is the *gas consumed* (in gas units) by each on-chain transaction, measured directly on the local Anvil L1. Gas units are deterministic and independent of network conditions, which makes them the reproducible basis for comparison. To give an intuitive sense of real-world cost, these gas units are additionally converted into a single euro figure at one clearly stated reference price (a representative mainnet gas price in gwei and a fixed ETH/EUR rate, both stated in the thesis). Since cost scales linearly with the gas price, the reader can rescale this figure to any other price.

Metric 3 — Sustained throughput. At each of the four target frequencies — 1 ms, 10 ms, 100 ms, 1 s (i.e., target rates of 1000, 100, 10, and 1 transactions per second respectively) — the protocol is driven for a fixed time window, and the achieved transactions-per-second is compared to the target. The protocol's sustained-throughput ceiling for a given workload is the highest target rate it actually sustains. For Perun the natural limit is the round-trip time of off-chain message exchange between the two channel peers (maximum sustained rate $\approx 1 / \mathrm{RTT}$). For the rollups, the limit is the local sequencer's own processing rate on the workstation.

4.4 Hardware and setup

Measurements run on a single desktop workstation (x86_64, 32 GB RAM). Possibly a Raspberry Pi cluster (Pi 3/4/5) will additionally be used to run the protocols on resource-constrained

hardware, in line with Kuehn's IoT-oriented setup [3]. To eliminate public-testnet variance — gas-price drift, sequencer queue fluctuations, RPC rate-limiting — all three protocols share the same locally hosted L1: a Foundry Anvil instance whose block production is driven on-demand by the workload generator at the frequency under test.

- **Perun:** two `go-perun` peer instances on the host, against an EVM adjudicator contract deployed on the local Anvil.
- **Arbitrum Nitro:** local `nitro-testnode` stack (sequencer, batch-poster, validator), configured to use the shared Anvil as parent chain.
- **zkSync Era:** local `zkstack` ecosystem pointed at the shared Anvil, the prover disabled. Proof generation requires hardware ($\geq$64 GB RAM, dedicated GPU) that is out of scope for this thesis; consequently, proof-verification latency is reported from protocol documentation rather than measured locally, in the same manner as Arbitrum's challenge-window duration. The validity-proof step itself is not on the per-transaction critical path of the comparison, so this scoping choice does not affect Metrics 1–3 as defined.
- **Workload generator:** a single Python script, parameterised by (use case, frequency, duration, payload-size), drives the protocol-under-test with a deterministic transaction sequence and is reused unchanged across all three protocols.
- **Repetitions:** each measurement (protocol $\times$ use case $\times$ frequency) is repeated across 3 runs, to capture residual variance.
- **Reproducibility:** all scripts, contract sources, Anvil and L2-stack configs, and raw measurement logs are versioned in a public Git repository and referenced from the thesis.

5 Deliverable and Timeline

The thesis is structured as follows:

1. Introduction.
2. Background on L1 vs L2 and data availability.
3. Selected protocols — Perun, Arbitrum Nitro, zkSync Era.
4. Comparison framework and metrics.
5. Workload 1: IoT micropayments.
6. Workload 2: On-chain token gaming.
7. Synthesis (the comparison table and workload-specific recommendations).
8. Conclusion.

A reproducibility artifact (workload generators, deployed contract addresses, raw measurement logs) is versioned in a public Git repository.

The work is planned over three months from registration:

- **Month 1** — write the Introduction and Background chapters; set up the shared Anvil L1; deploy the Perun adjudicator contract; set up the Arbitrum Nitro testnode and zkSync Era local stack against the shared L1; implement the workload generator.
- **Month 2** — collect measurements for UC1 (IoT micropayments) and UC2 (two-player gaming) across all three protocols and the full frequency sweep; build the synthesis matrix; draft the two workload chapters and the synthesis chapter.
- **Month 3** — complete the remaining chapters (Selected Protocols, Comparison Framework, Conclusion); integrate supervisor feedback; polish; prepare the final presentation.

References

[1] M. B. Saif, S. Migliorini, and F. Spoto, "A survey on data availability in Layer 2 blockchain rollups: Open challenges and future improvements," *Future Internet*, vol. 16, no. 9, p. 315, 2024.

[2] S. Dziembowski, S. Faust, and K. Hostáková, "General state channel networks," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018. Cryptology ePrint Archive 2018/320.

[3] U. A. Kuehn, "A decentralized IoT payment service for digital currencies," in *2025 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*, (Madison, WI, USA), pp. 1–4, 2025.

[4] L. D. Negka and G. P. Spathoulas, "Blockchain state channels: A state of the art," *IEEE Access*, vol. 9, pp. 160277–160298, 2021.

[5] G. A. F. Rebello, G. F. Camilo, L. A. C. de Souza, M. Potop-Butucaru, M. Dias de Amorim, M. E. M. Campista, and L. H. M. K. Costa, "A survey on blockchain scalability: From hardware to layer-two protocols," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 4, pp. 2411–2458, 2024.

[6] H. S. Galal, M. ElSheikh, and A. M. Youssef, "An efficient micropayment channel on Ethereum," in *Data Privacy Management, Cryptocurrencies and Blockchain Technology* (C. Pérez-Solà, G. Navarro-Arribas, A. Biryukov, and J. Garcia-Alfaro, eds.), pp. 211–218, Springer, 2019.

[7] A. Gangwal, H. R. Gangavalli, and A. Thirupathi, "A survey of layer-two blockchain protocols," *Journal of Network and Computer Applications*, vol. 209, p. 103539, 2022.

[8] C. Sguanci, R. Spatafora, and A. M. Vergani, "Layer 2 blockchain scaling: a survey," 2021.

[9] Z. Avarikioti, M. Maffei, and Y. Wang, "A security framework for general blockchain layer 2 protocols," 2025.